



Understand the storage class

This lab must be done on Unix-based OS

1. Static Storage Class

Step 1: Compile and run this program **3 times**, copy the output and put it in your report, explain why the result show up that way.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void main()
5  {
6      auto int x = 3;
7
8      while (x > 0)
9      {
10         static int y = 5;
11         y++;
12
13         printf("The value of y is %d\n", y);
14         printf("The address of y is %p\n\n", &y);
15
16         x--;
17     }
18 }
```

Step 2: Modify the code by delete the “static” word, compile, and re-run the program.

Step 3: Copy the output and put it in your report, explain why the results show up that way, and discuss the difference between both outputs.

Step 4: Redo step 1-3, but this time compile the program using:

“gcc -no-pie -o <your executable object name> <your c code name>”

Are they different? Why or why not

2. Extern Storage Class

Step 1: Compile and run this program **3 times**, copy the output and put it in your report, explain why the result show up that way.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int x = 20;
5
6  void display()
7  {
8      extern int x;
9      printf(" Display value of x: %d\n", x);
10     printf(" Adress of x (in display function): %p\n\n", &x);
11 }
12
13 void main()
14 {
15     extern int x;
16     printf(" Print value of x: %d\n", x);
17     printf(" Adress of x (in main function): %p\n\n", &x);
18
19     display();
20 }
```

Step 2: Modify the code by delete the “extern” word **in main() function (line 15)**, compile, and re-run the program.

Step 3: Copy the output and put it in your report, explain why the results show up that way, and discuss the difference between both outputs.

Step 4: Redo step 1-3, but this time compile the program using:

“gcc -no-pie -o <your executable object name> <your c code name>”

Are they different? Why or why not

3. Understand the memory allocation function by following those steps.

Step 1: Run this program (without '-no-pie'):

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void main()
5  {
6      int *a;
7      float *b;
8      int c[10];
9
10     printf("\nAddress of Pointer\n");
11     printf(">>%p\n", &a);
12     printf(">>%p\n", &b);
13
14     printf("\nEffective Address\n");
15     printf(">>%p\n", a);
16     printf(">>%p\n", b);
17
18     a = (int*) malloc(10*sizeof(int));
19     printf("\nAfter malloc Pointer a\n");
20     printf(">>%p\n", a);
21     printf(">>%p\n", &a[0]);
22     printf(">>%p\n", &a[9]);
23
24     printf("\nArray c\n");
25     printf(">>%p\n", c);
26     printf(">>%p\n", &c[0]);
27     printf(">>%p\n", &c[9]);
28
29     /*    b = (float*) malloc(10*sizeof(float));
30     printf("\nAfter malloc Pointer b\n");
31     printf(">>%p\n", b);
32     printf(">>%p\n", &b[0]);
33     printf(">>%p\n", &b[9]);
34     */
35
36     a = (int*) realloc(a, 1000*sizeof(int));
37     printf("\nAfter realloc Pointer a\n");
38     printf(">>%p\n", a);
39     printf(">>%p\n", &a[0]);
40     printf(">>%p\n", &a[9]);
41     printf(">>%p\n", &a[999]);
42
43     free(a);
44     // free(b);
45 }
```

Step 2: Copy the output and put it in your report

step 3: Explain, for each section (Address of pointer, EA, array a after malloc(), and array c) what are their meanings? Why do the results show up that way?

step 4: Modify the code by uncomment 2 places, and re-run this program

step 5: Copy the output and put it in your report

step 6: Explain why the result show up that way? Is the result different from the previous run? Especially for the address of array a. Why or why not?

Note: all steps must be executed in the same machine.

Submission

Submit the report into LEB2 before the due date. This is the problem session for the group of up to 5 students.